

Guia de Estudos: Sistemas Operacionais II

Instituto Federal Sul-rio-grandense - Câmpus Passo Fundo

Bacharelado em Ciência da Computação

Disciplina: Sistemas Operacionais II

Professor: Prof. Leonardo Deliyannis Constantin

Tópico: Problemas Clássicos de Sincronização e Concorrência

1. Fundamentação Teórica e Primitivas

A execução concorrente exige mecanismos para evitar que múltiplos processos acessem simultaneamente recursos compartilhados de forma inconsistente.

- **Condição de Corrida (Race Condition):** Situação onde o resultado final depende da ordem de execução das instruções no tempo.
- **Região Crítica:** Trecho de código que acessa recursos compartilhados. Apenas um processo pode estar nela por vez (**Exclusão Mútua**).
- **Semáforos:** Primitiva variável inteira acessada via:
 - **wait (P / sem_wait):** Decrementa o semáforo. Se < 0 , o processo bloqueia.
 - **signal (V / sem_post):** Incrementa o semáforo. Se houver processos bloqueados, um é acordado.
- **Condições de Coffman (Deadlock):** Para que ocorra um impasse total, quatro condições devem ser simultâneas:
 - a. **Exclusão Mútua:** Recurso não compartilhável.
 - b. **Posse e Espera:** Processo retém um recurso enquanto espera outro.
 - c. **Não Preempção:** Recursos não podem ser retirados à força.
 - d. **Espera Circular:** Cadeia fechada de processos esperando recursos uns dos outros.

2. Problema 1: Produtor-Consumidor (Buffer Limitado)

O desafio consiste em coordenar dois processos que compartilham um buffer de tamanho fixo N . O produtor não pode inserir em um buffer cheio (**overflow**) e o consumidor não pode retirar de um buffer vazio (underflow).

Solução com Semáforos

Utilizam-se três semáforos:

- **mutex**: Garante exclusão mútua no acesso ao buffer (inicializado em 1).
- **empty**: Conta espaços vazios (inicializado em N).
- **full**: Conta itens disponíveis (inicializado em 0).

Pseudo-código

```
// PRODUTOR
```

```
while(true) {
```

```
    item = produzir_item();
```

```
    wait(empty); // Espera por espaço livre
```

```
    wait(mutex); // Entra na região crítica
```

```
    inserir_item(item);
```

```
    signal(mutex); // Sai da região crítica
```

```
    signal(full); // Notifica que há um novo item
```

```
}
```

```
// CONSUMIDOR
```

```
while(true) {
```

```
    wait(full); // Espera por item disponível
```

```
wait(mutex); // Entra na região crítica

item = remover_item();

signal(mutex); // Sai da região crítica

signal(empty); // Notifica que liberou um espaço

processar_item(item);

}
```

Pegadinha de Prova: A inversão de `wait(empty)` com `wait(mutex)` causa **Deadlock**. Se o buffer estiver cheio, o produtor pegará o mutex e ficará bloqueado em `empty`. Como ele detém o mutex, o consumidor nunca conseguirá entrar para remover um item e liberar espaço.

3. Problema 2: Jantar dos Filósofos

Cinco filósofos sentam-se à mesa para pensar e comer. Para comer, precisam de dois garfos (esquerdo e direito). O problema clássico ilustra o risco de deadlock e starvation.

Estratégias de Resolução

1. **Assimetria:** Um filósofo (ex: o de índice ímpar) pega primeiro o garfo esquerdo e depois o direito, enquanto o de índice par faz o inverso. Isso quebra a **Espera Circular**.
2. **Solução de Tanenbaum (Estados):** Define estados `PENSANDO`, `FOME` e `COMENDO`. Um filósofo só começa a comer se ambos os vizinhos não estiverem comendo.
3. **Árbitro/Limitador:** Permitir que apenas $N-1$ filósofos entrem na sala de jantar simultaneamente, garantindo que sempre haverá ao menos um filósofo com dois garfos disponíveis.

4. Problema 3: Barbeiro Adormecido

Um barbeiro trabalha em uma barbearia com uma cadeira de corte e N cadeiras de espera. Se não há clientes, o barbeiro dorme. Se um cliente chega e o barbeiro está dormindo, ele o acorda. Se o barbeiro está ocupado, o cliente senta se houver cadeira livre; caso contrário, vai embora (desistência).

Pseudo-código (Sincronização)

```
// BARBEIRO
```

```
while(true) {
```

```
    wait(clientes_esperando); // Dorme se não houver clientes
```

```
    wait(mutex);           // Protege acesso ao contador
```

```
    num_espera--;
```

```
    signal(barbeiro_pronto); // Pronto para cortar
```

```
    signal(mutex);
```

```
    cortar_cabelo();
```

```
}
```

```
// CLIENTE
```

```
wait(mutex);
```

```
if (num_espera < N) {
```

```
    num_espera++;
```

```
    signal(clientes_esperando); // Avisa o barbeiro
```

```
    signal(mutex);
```

```
    wait(barbeiro_pronto); // Espera sua vez na cadeira
```

```
receber_corte();

} else {

    signal(mutex);        // Vai embora (buffer cheio)

}
```

5. Tabela Comparativa

Problema	Primitivas Típicas	Risco Primário	Técnica Chave
Produtor-Consumidor	2 Contadores, 1 Mutex	Deadlock (inversão)	Ordem estrita dos <code>wait()</code>
Jantar dos Filósofos	Vetor de Semáforos	Deadlock / Starvation	Quebra da Espera Circular
Barbeiro Adormecido	Mutex e Sem. Binários	Race Condition no contador	Sincronização Cliente-Servidor

Analogias Visuai:

- **Produtor:** Uma linha de montagem de caixas.
- **Barbeiro:** Uma sala de espera de consultório com apenas uma vaga no médico.
- **Filósofos:** Recursos de hardware (impressoras) sendo disputados por processos.